

Notebook

April 28, 2026

1 Evaluation: Scenario-Based vs Microbenchmark Benchmarking

Thesis context: Meaningful performance evaluation of serverless applications requires benchmarks that reflect realistic usage patterns rather than isolated micro-level measurements. This notebook evaluates whether deriving workload scenarios from OpenAPI specifications produces richer, more actionable performance insights than traditional microbenchmark-style load testing with wrk2.

Testing Points: - **TP-A:** Per-Operation Performance Characterization (insight depth + saturation detection + data validity) - **TP-B:** Execution Phase Granularity (cold vs steady per-operation penalties) - **TP-C:** Scenario Shape Sensitivity (read-heavy vs mixed vs lifecycle) - **TP-D:** Cross-Framework Decision Quality (Spring vs Quarkus)

1.1 Methodology

1.1.1 Measurement Semantics

The two treatments measure fundamentally different quantities:

	wrk2 (microbenchmark)	slsbench (scenario-based)
Latency reported	Flow completion time – the total round-trip of the entire multi-step request chain (all operations combined)	Per-step latency – individual HTTP request duration for each operation in the chain
Percentiles (p50/p99)	Computed over flow-completion times	Computed over individual request elapsed times (from <code>step_latencies_thread_*.jsonl</code>)
Error accounting	Non-2xx count in wrk2 summary	Per-operation status codes from <code>step_status</code> in <code>flow_stats</code>
Body generation	Hardcoded IDs in Lua scripts	Stateful Schemathesis-generated chains with valid payloads per iteration

Because wrk2 measures the sum of all steps while slsbench measures each step individually, their absolute latency numbers are **not directly comparable**. The analysis uses each where appropri-

ate: wrk2 aggregates for overall throughput/error comparison, slsbench per-step for operation-level insights.

1.1.2 Hypothesis Confirmation Thresholds

Threshold	Value	Rationale
Variance (TP-A)	max/min > 2x	Conservative; even a 2x spread between operations is invisible in wrk2 aggregates
Saturation spread (TP-A)	jump ratio spread > 1.5x	Operations degrading at different rates under load
Cold penalty spread (TP-B)	max/min penalty > 1.5x	Non-uniform cold penalties across operations
Scenario sensitivity (TP-C)	mean CV > 0.15	Meaningful latency variation across scenario shapes
Error rate (TP-A)	wrk2 > 0.5%	Elevated errors from stale hardcoded IDs

1.1.3 Scenario Derivation from OpenAPI Specifications

The scenario-based approach derives workload flows from the application’s OpenAPI description in three steps:

1. **OpenAPI Link Discovery:** The OpenAPI specification defines **links** between operations. For example, `addOwner` (POST `/api/owners`) declares links to `getOwner`, `addPetToOwner`, `addVet`, etc. via `$response.body#/id` parameter passing. These links encode the stateful transitions that real API consumers follow.
2. **Flow DSL Construction:** Using the discovered links, a flow YAML is constructed that models a realistic usage pattern. For example, the *lifecycle* scenario chains: `addOwner` -> `addPetToOwner` (35%) / `getOwner` (25%) / `listPets` (15%) / `addVet` (15%) / `listVets` (10%), with deeper chains like `addPetToOwner` -> `addVisit` -> `getVisit`.
3. **Stateful Body Generation (probe-bodies):** The `slsbench probe-bodies` command uses `schemathesis` to generate valid request bodies for each chain step. It executes the chains against the running application, following OpenAPI links to resolve IDs and parameters, producing reusable iteration files (e.g., `iteration-001.json`) that contain complete request sequences with valid, interdependent data.

This pipeline (OpenAPI spec -> flow DSL -> probe-bodies -> benchmark execution) ensures that benchmarks exercise realistic, stateful request chains rather than isolated endpoint hits.

1.1.4 Data Inventory

The data inventory below is auto-detected from available results. Error bars and confidence intervals require $n \geq 3$ repetitions.

1.1.5 Measurement Setup

- **Steady-state duration:** 30s (with 10s warmup stage before measurement)

- **Cold-start duration:** 30s (no warmup; container started fresh)
- **wrk2 config:** 2 threads, 5 connections
- **Network:** `--network=host` (Docker) to minimize network overhead
- **Host:** Single machine, all experiments sequential, no concurrent workloads
- **Spring DB:** H2 in-memory | **Quarkus DB:** PostgreSQL (Docker container)
- **Frameworks:** Spring Boot (JVM), Quarkus JVM, Quarkus Native (GraalVM)

Results root: `/home/bakhtia/Documents/thesis/harness-evaluation/evaluation/results`

1.2 0. Data Loading & Parsing

Auto-discover all benchmark results under `results/`. Each result lives at:

`results/<app>/<treatment>/<scenario>/<phase>/R<rate>/run-<N>/`

Discovered 346 result directories

	treatment	app	scenario	phase	runs
0	slsbench	go	lifecycle	cold	12
1	slsbench	go	lifecycle	steady	12
2	slsbench	go	mixed	cold	12
3	slsbench	go	mixed	steady	12
4	slsbench	go	read-heavy	cold	12
5	slsbench	go	read-heavy	steady	12
6	slsbench	quarkus	lifecycle	cold	6
7	slsbench	quarkus	lifecycle	steady	6
8	slsbench	quarkus	mixed	cold	6
9	slsbench	quarkus	mixed	steady	6
10	slsbench	quarkus	read-heavy	cold	5
11	slsbench	quarkus	read-heavy	steady	6
12	slsbench	quarkus-jvm	lifecycle	cold	4
13	slsbench	quarkus-jvm	lifecycle	steady	4
14	slsbench	quarkus-jvm	mixed	cold	6
15	slsbench	quarkus-jvm	mixed	steady	4
16	slsbench	quarkus-jvm	read-heavy	cold	6
17	slsbench	quarkus-jvm	read-heavy	steady	5
18	slsbench	spring	lifecycle	cold	6
19	slsbench	spring	lifecycle	steady	6
20	slsbench	spring	mixed	cold	6
21	slsbench	spring	mixed	steady	6
22	slsbench	spring	read-heavy	cold	6
23	slsbench	spring	read-heavy	steady	6
24	wrk2	go	lifecycle	cold	12
25	wrk2	go	lifecycle	steady	12
26	wrk2	go	mixed	cold	12
27	wrk2	go	mixed	steady	12
28	wrk2	go	read-heavy	cold	12
29	wrk2	go	read-heavy	steady	12
30	wrk2	quarkus	lifecycle	cold	6

31	wrk2	quarkus	lifecycle	steady	6
32	wrk2	quarkus	mixed	cold	6
33	wrk2	quarkus	mixed	steady	6
34	wrk2	quarkus	read-heavy	cold	6
35	wrk2	quarkus	read-heavy	steady	6
36	wrk2	quarkus-jvm	lifecycle	cold	4
37	wrk2	quarkus-jvm	lifecycle	steady	4
38	wrk2	quarkus-jvm	mixed	cold	6
39	wrk2	quarkus-jvm	mixed	steady	4
40	wrk2	quarkus-jvm	read-heavy	cold	6
41	wrk2	quarkus-jvm	read-heavy	steady	6
42	wrk2	spring	lifecycle	cold	6
43	wrk2	spring	lifecycle	steady	6
44	wrk2	spring	mixed	cold	6
45	wrk2	spring	mixed	steady	6
46	wrk2	spring	read-heavy	cold	6
47	wrk2	spring	read-heavy	steady	6

wrk2 aggregate rows: 174
 slsbench per-step rows: 2173
 slsbench aggregate rows: 367
 first-response rows: 260
 container-stats rows: 342
 raw latency records: 1269272

slsbench step columns: ['app', 'treatment', 'scenario', 'phase', 'rate', 'run',
 'path', 'stage', 'operation_id', 'avg_us', 'max_us', 'sum_us', 'count',
 'responses_2xx', 'responses_non2xx', 'p50_us', 'p95_us', 'p99_us',
 'responses_2xx_raw', 'responses_non2xx_raw']

app	treatment	scenario	phase	rate	run	
path	stage	operation_id	avg_us	max_us	\	
0	go	slsbench	lifecycle	cold	1000	1
../results/go/slsbench/lifecycle/cold/R1000/run-1						lifecycle addOwner
1567	16483					
1	go	slsbench	lifecycle	cold	1000	1
../results/go/slsbench/lifecycle/cold/R1000/run-1						lifecycle addPetToOwner
1614	13345					
2	go	slsbench	lifecycle	cold	1000	1
../results/go/slsbench/lifecycle/cold/R1000/run-1						lifecycle addVet
2266	14807					

	sum_us	count	responses_2xx	responses_non2xx	p50_us	p95_us	p99_us
responses_2xx_raw			responses_non2xx_raw				
0	5401812	3447	3447		0	999.0	4798.20
	3447.0		0.0				7665.08
1	1936149	1199	1199		0	1111.0	4431.30
	1199.0		0.0				7233.62
2	1169724	516	328	188	1630.5	7125.25	11078.45

328.0

188.0

Normalized 5 Quarkus operation names to Spring equivalents for cross-framework comparison.

1.2.1 Data Inventory (auto-detected)

Dimension	Value
Frameworks	go, quarkus, quarkus-jvm, spring (4)
Scenarios	lifecycle, mixed, read-heavy (3)
Phases	cold, steady (2)
Rates	50, 200, 500, 1000 req/s
Max repetitions	3 per config
Total result dirs	346

1.2.2 Runtime-class coverage

	runtime_class	framework	app	runs
0	graalvm-native	quarkus	quarkus	71
1	jvm	quarkus	quarkus-jvm	59
2	jvm	spring	spring	72
3	native-nonjvm	go	go	144

1.3 TP-A: Per-Operation Performance Characterization

Hypothesis: OpenAPI-derived scenario benchmarks reveal per-operation performance variance and per-operation saturation points that aggregate microbenchmarks mask.

This TP merges three claims: 1. Per-operation latency variance is hidden by aggregates (>10x spread) 2. Under increasing load, operations saturate at different rates 3. wrk2 hardcoded IDs produce elevated error rates (data validity)

Table A1: slsbench Per-Operation Latency Breakdown (lifecycle, R=1000, steady, n=3 run(s))

Active operations: 11 (excluded 0 with zero traffic)

Max/Min variance ratio: 7.8x

operation_id	Avg (us)	Std Dev	p50 (us)	p95 (us)	p99 (us)	N	% of Flow
listPets	2899.3	58.1	1817.0	8144.2	13758.8	1471	19.3
addVet	2629.0	92.0	2066.2	7598.9	10627.3	1467	17.5
listVets	2016.3	149.7	1055.8	6777.4	10756.9	994	13.4
addPetToOwner	1910.7	36.1	1233.7	5436.7	8266.6	3402	12.7
addOwner	1704.7	9.5	1007.2	5353.5	8005.7	9803	11.3
addVisitToOwner	1424.0	52.9	859.3	4670.4	6442.8	1725	9.5
listVisits	707.0	4.6	345.7	2900.7	5272.7	683	4.7
getVet	530.0	87.5	118.0	2706.4	4946.1	944	3.5
getVisit	434.3	47.8	134.5	1806.3	4560.8	1042	2.9

getOwnersPet	423.0	15.9	122.0	1868.6	4633.0	1677	2.8
getOwner	373.7	7.5	121.0	1468.9	4163.9	2463	2.5

wrk2 aggregate *flow completion time* at R=1000 (per framework):
 go: 17,023,333 us (17,023.3 ms), achieved 148 rps [SATURATED]
 Note: frameworks marked [SATURATED] achieved <70% of target rate (1000 rps).
 Their latency reflects queuing backlog, not pure application speed.

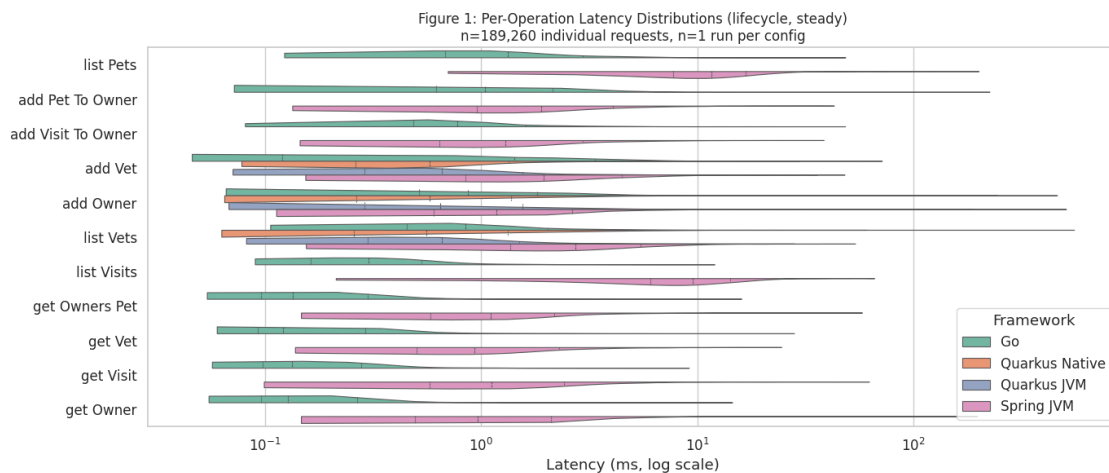
wrk2 reports ONE number per framework; slsbench reveals 11 distinct per-step latencies
 ranging from 374 to 2899 us

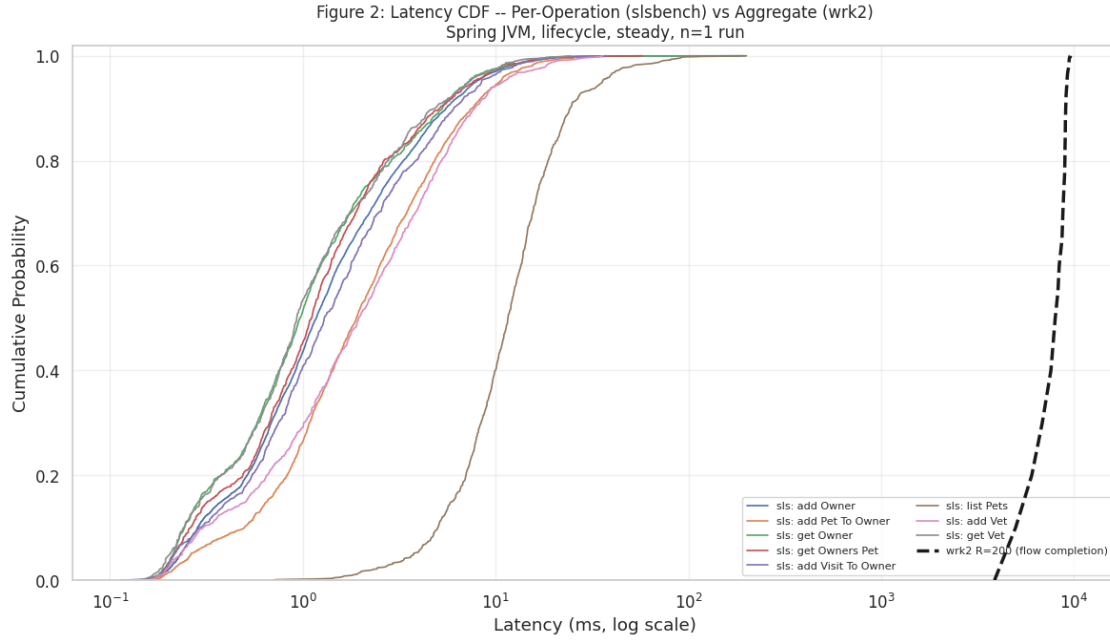
/tmp/ipykernel_18898/1119891326.py:21: FutureWarning:

The `scale` parameter has been renamed and will be removed in v0.15.0. Pass
 `density\_norm='width'` for the same effect.
 sns.violinplot(

/tmp/ipykernel_18898/1119891326.py:36: UserWarning: set_ticklabels() should only
 be used with a fixed number of ticks, i.e. after set_ticks() or using a
 FixedLocator.

ax.set_yticklabels([wrapped_operation_label(op, width=18) for op in op_order])





Note: wrk2 CDF spans 3869--9568 ms (flow completion time, all steps combined). slsbench CDFs span per-step latencies. The scale difference reflects the measurement semantics.

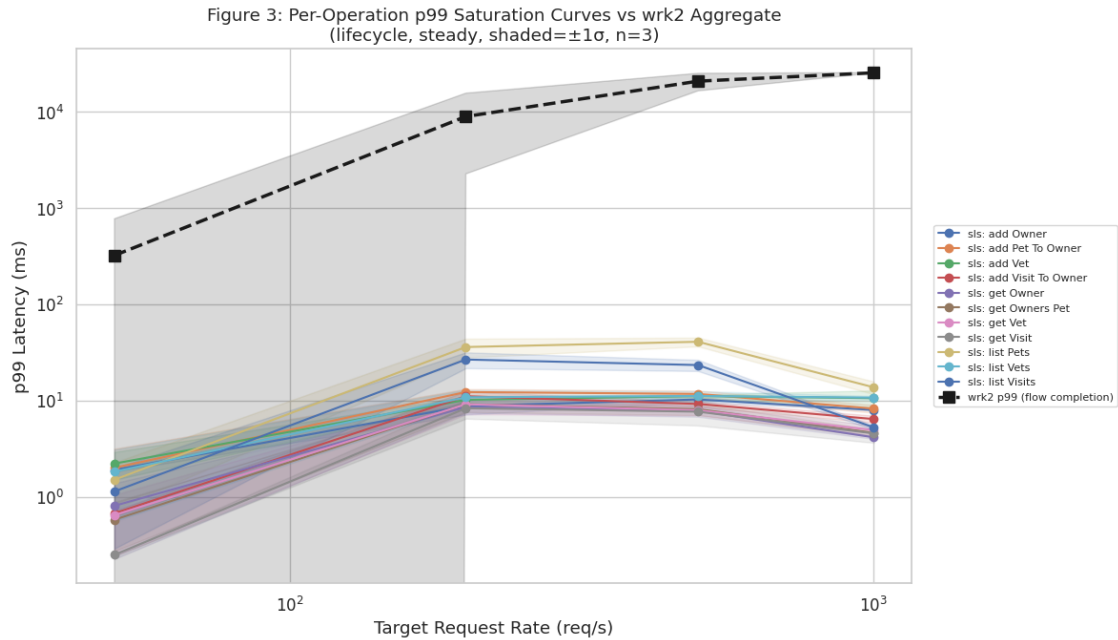


Table A3: Data Validity Comparison (lifecycle, steady)

Treatment	Error Rate (%)		Non-2xx		Total Requests	
	slsbench	wrk2	slsbench	wrk2	slsbench	wrk2
Rate						
50	1.88	0.00	84	0	4460	1502
200	0.86	4.42	467	189	54613	4280
500	0.59	7.09	620	322	104516	4552
1000	2.03	0.00	522	0	25671	4455

1.3.1 TP-A Hypothesis Results

Variance: max/min = 6.7x (threshold >2.0x) – **CONFIRMED**

Fastest: `getVet` = 904 us | Slowest: `listPets` = 6104 us

Saturation: jump spread = 7.9x (threshold >1.5x) – **CONFIRMED**

Top degraders: `getVisit` 34.6x, `listPets` 23.2x, `listVisits` 20.4x

Validity: wrk2 error rate = 5.30% (threshold >0.5%) – **CONFIRMED**

slsbench error rate = 0.89%

1.4 TP-B: Execution Phase Granularity

Hypothesis: Cold-start penalties differ per-operation, and scenario-based benchmarks expose this while aggregate wrk2 measurements report a single blended value.

Independent variable: Execution phase (cold vs steady)

Control: scenario=mixed, rate=500, app=spring

Per-framework cold penalty (mixed scenario):

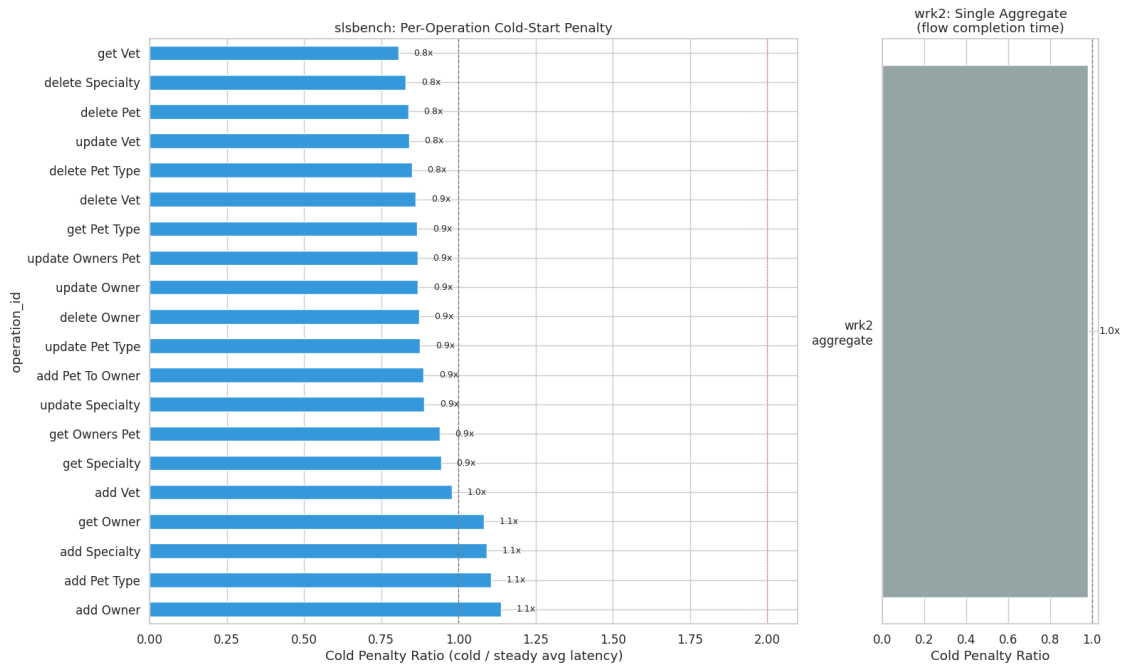
go: penalty range 0.8x to 1.1x (mean 1.0x)

quarkus: penalty range 1.0x to 1.1x (mean 1.1x)

quarkus-jvm: penalty range 1.0x to 1.0x (mean 1.0x)

spring: penalty range 0.9x to 1.3x (mean 1.1x)

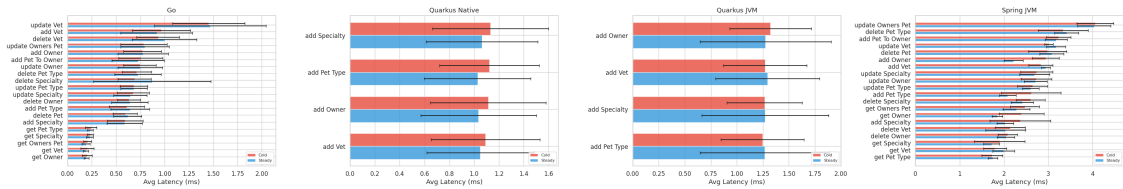
Figure 4: Cold-Start Penalty -- Per-Op (slsbench) vs Aggregate (wrk2)



Per-operation cold penalty range: 0.8x to 1.1x
Operations with penalty >2x: 0 of 20

Note: per-operation avg_us is amortized over the full 30s measurement window.
The cold-start impact (first few seconds) is diluted by hundreds of subsequent warmed-up requests. See first-response times below for true cold-start behavior.

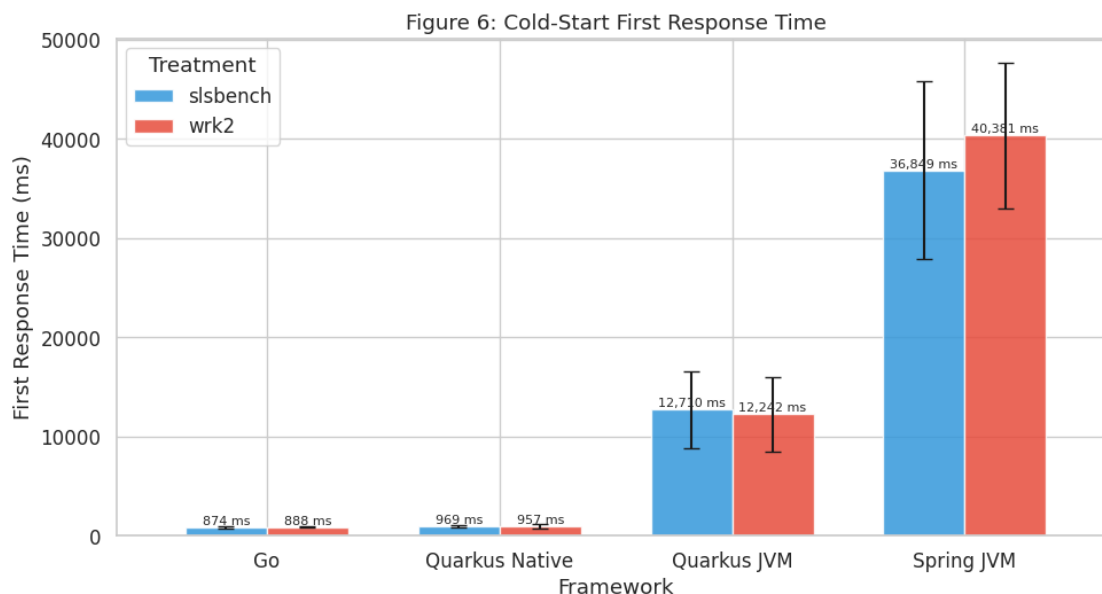
Figure 5: Absolute Latency -- Cold vs Steady per Operation (mixed scenarios, error bars=1s, ms)



First-Response Time Comparison (cold phase)

		mean_ms	std_ms	count
treatment app	go	873.5	106.9	36
	quarkus	968.5	135.9	17
	quarkus-jvm	12709.6	3871.0	16
	spring	36848.6	8953.3	18

wrk2	go	888.5	49.7	36
	quarkus	956.7	177.3	18
	quarkus-jvm	12242.2	3728.8	16
	spring	40380.9	7342.7	18



First-response interpretation: go starts 42x faster than spring (874 ms vs 36,849 ms). Quarkus Native vs JVM: 969 ms vs 12,710 ms (13x). This confirms that runtime configuration dominates cold-start behavior, while per-operation penalties (Chart B1) are amortized over the full test window.

wrk2 Aggregate: cold vs steady (mixed scenario)

Note: wrk2 latencies represent flow completion times, not individual request latency.

	avg_latency_ms	p99_latency_ms	throughput	non_2xx
phase				
cold	9045.35	13867.03	154.92	3189
steady	9212.41	14074.17	134.33	4831

1.4.1 TP-B Hypothesis Results

(a) **Per-operation amortized penalty spread:** max=1.1x, min=0.8x, spread=1.4x (threshold >1.5x) – **NOT CONFIRMED**

Note: avg_us is amortized over the full 30s measurement window; cold-start impact is diluted by hundreds of warmed-up requests.

wrk2 cold avg: 9045 ms | steady avg: 9212 ms | ratio: 1.0x (single aggregate, flow completion time)
slsbench reveals 20 distinct penalties: 0.8x – 1.1x

(b) **First-response spread across runtimes:** max=36,849 ms (spring), min=874 ms (go), ratio=42x – **CONFIRMED**

First-response time captures the true cold-start penalty before JVM warmup dilutes it.

Combined verdict: CONFIRMED – scenario-based benchmarking reveals execution-phase granularity that aggregates mask (per-op penalty variability + dramatic first-response differences).

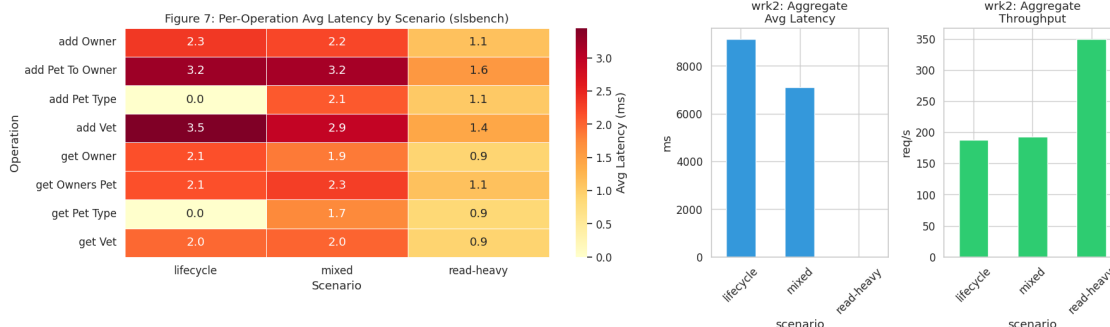
1.5 TP-C: Scenario Shape Sensitivity

Hypothesis: Different scenario shapes (read-heavy vs mixed vs lifecycle) produce measurably different per-operation latency profiles, while wrk2 reports similar aggregates across all scenarios.

Independent variable: Scenario (read-heavy, mixed, lifecycle)

Control: app=spring, phase=steady, rate=500

Figure 7: Scenario Shape Sensitivity -- slsbench per-op detail vs wrk2 aggregate



Coefficient of Variation per operation (ops present in all 3 scenarios):

addVet: CV = 0.404

getOwner: CV = 0.393

getVet: CV = 0.376

addOwner: CV = 0.353

addPetToOwner: CV = 0.349

getOwnersPet: CV = 0.343

Figure 8: Operation Presence Across Scenarios
25 total operations, 6 shared across all

add Owner	1	1	1
add Pet To Owner	1	1	1
add Vet	1	1	1
get Owner	1	1	1
get Owners Pet	1	1	1
get Vet	1	1	1
get Pet Type	0	1	1
add Pet Type	0	1	1
add Specialty	0	1	0
delete Pet Type	0	1	0
delete Pet	0	1	0
delete Owner	0	1	0
add Visit To Owner	1	0	0
delete Specialty	0	1	0
delete Vet	0	1	0
get Specialty	0	1	0
get Visit	1	0	0
list Pets	1	0	0
list Vets	1	0	0
list Visits	1	0	0
update Owner	0	1	0
update Owners Pet	0	1	0
update Pet Type	0	1	0
update Specialty	0	1	0
update Vet	0	1	0
	lifecycle	mixed Scenario	read-heavy

Operations shared across all 3 scenarios: 6/25

lifecycle: 11 operations (5 unique to this scenario)

mixed: 20 operations (12 unique to this scenario)

read-heavy: 8 operations (0 unique to this scenario)

Key insight: Scenario shapes produce structurally different workloads -- different operations are exercised, not just different latencies for the same operations.

This structural difference is the primary form of scenario sensitivity.

LATENCY SENSITIVITY: Mean CV = 0.369 (threshold: >0.15)

-> CONFIRMED: shared operations have significantly different latencies across scenarios

STRUCTURAL SENSITIVITY: 6/25 operations shared (24%)

-> CONFIRMED: scenarios exercise substantially different operation sets

COMBINED VERDICT: CONFIRMED

wrk2 aggregate CV: 0.885

slsbench per-op mean CV: 0.369

1.6 TP-D: Cross-Framework Decision Quality

Hypothesis: Per-operation metrics from scenario-based benchmarks enable more informed framework selection than aggregate wrk2 latency numbers.

Independent variable: Framework (Spring, Quarkus JVM)

Control: scenario=lifecycle, phase=steady, rate=500

Table D1: slsbench Per-Operation Avg Latency by Framework (lifecycle, steady)

	go (us)	quarkus (us)	quarkus-jvm (us)	spring (us)
operation_id				
addOwner	1247.7	1157.7	1380.8	2339.7
addPetToOwner	1438.2	NaN	NaN	3234.8
addVet	1795.4	1089.3	1324.2	3450.3
addVisitToOwner	1129.5	NaN	NaN	2503.5
getOwner	334.0	NaN	NaN	2143.7
getOwnersPet	372.8	NaN	NaN	2143.3
getVet	368.7	NaN	NaN	1975.8
getVisit	322.1	NaN	NaN	2300.0
listPets	1843.0	NaN	NaN	14624.8
listVets	1380.8	1115.0	1297.0	4459.7
listVisits	542.7	NaN	NaN	11077.5

Operation-level winners (3 shared operations):

go: fastest on 0/3 operations

quarkus: fastest on 3/3 operations

quarkus-jvm: fastest on 0/3 operations

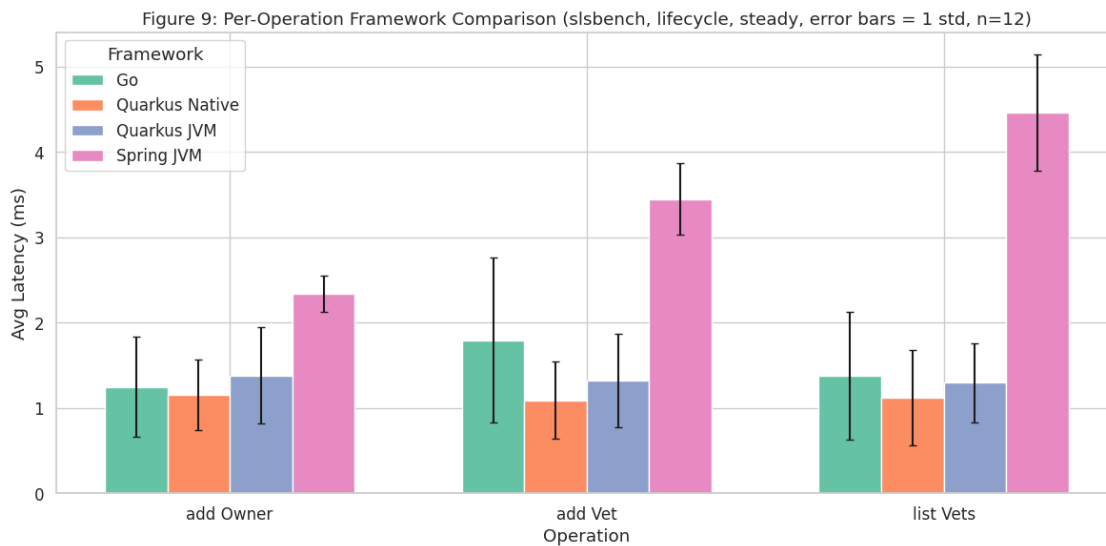
spring: fastest on 0/3 operations

Table D2: wrk2 Aggregate Comparison by Framework

Note: wrk2 latency = flow completion time (all chain steps combined)

	avg_latency_ms	p99_ms	throughput_rps	non_2xx
app				
go	7528.38	11441.48	142.24	0
quarkus	11625.00	18822.50	95.21	0
quarkus-jvm	14157.50	21225.00	81.38	0

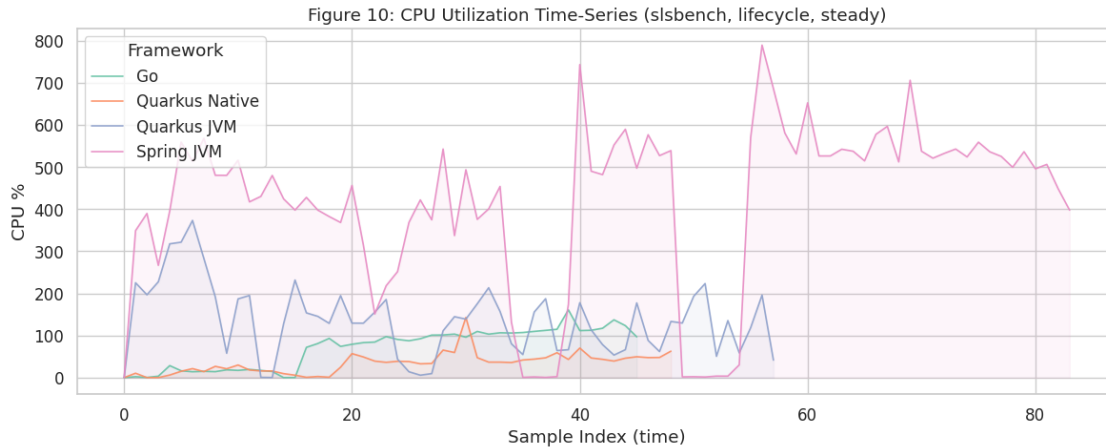
spring 9133.33 12856.67 187.70 5632



Container Resource Utilization Summary:

				avg_cpu	max_cpu	avg_mem	n_runs
treatment	app	scenario	phase				
slsbench	go	lifecycle	cold	60.8	161.3	0.1	3
			steady	51.9	160.9	0.1	3
		mixed	cold	35.7	84.4	0.1	3
			steady	30.5	184.8	0.1	3
		read-heavy	cold	38.5	84.6	0.1	3
			steady	31.4	97.9	0.1	3
	quarkus	lifecycle	cold	57.2	294.9	0.3	3
			steady	66.9	346.3	0.3	3
		mixed	cold	92.0	298.3	0.3	3
			steady	71.2	305.5	0.3	3
		read-heavy	cold	60.8	320.8	0.3	3
			steady	26.7	129.1	0.3	3
	quarkus-jvm	lifecycle	cold	139.6	399.9	1.6	2
			steady	218.2	638.1	1.6	2
		mixed	cold	255.8	660.4	1.6	3
			steady	215.8	733.2	1.5	2
	spring	read-heavy	cold	278.9	883.8	1.7	3
			steady	213.7	709.4	1.6	3
		lifecycle	cold	467.2	1177.6	2.1	3
			steady	420.4	1494.7	2.2	3
		mixed	cold	436.4	1508.3	2.1	3
			steady	427.5	1046.2	2.2	3
		read-heavy	cold	376.3	842.0	2.2	3
			steady	271.3	907.8	2.4	3

wrk2	go	lifecycle	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		mixed	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		read-heavy	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
	quarkus	lifecycle	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	2
		mixed	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		read-heavy	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
	quarkus-jvm	lifecycle	cold	NaN	NaN	NaN	2
			steady	NaN	NaN	NaN	2
		mixed	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	2
		read-heavy	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	2
	spring	lifecycle	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		mixed	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		read-heavy	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3



1.6.1 TP-D Hypothesis Results

Per-operation comparison: 3 shared operations across 4 frameworks

- go fastest on 0/3 operations
- quarkus fastest on 3/3 operations
- quarkus-jvm fastest on 0/3 operations
- spring fastest on 0/3 operations

CONFIRMED: quarkus dominates, but per-op data reveals margin variability.

wrk2 aggregate winner: go (flow completion time)

wrk2 provides 1 ranking; slsbench reveals 3 operation-level rankings across 4 frameworks.

Operation	Best	go (us)	quarkus (us)	quarkus-jvm (us)	spring (us)
addOwner	quarkus	1,248	1,158	1,381	2,340
addVet	quarkus	1,795	1,089	1,324	3,450
listVets	quarkus	1,381	1,115	1,297	4,460

1.7 5. Summary and Thesis Conclusions

This section maps the experimental results to the thesis abstract claims and provides a structured hypothesis confirmation summary.

Abstract Claim	Testing Point	Key Evidence
Realistic usage patterns > isolated micro-measurements	TP-A	Per-operation variance, saturation curves, error rates
Model execution phases (cold start, steady state)	TP-B	Per-operation cold penalty ratios vs wrk2 single aggregate
Different scenario shapes yield different insights	TP-C	Latency heatmap CV across scenarios vs wrk2 flat aggregates
Comparative insights across frameworks/runtimes	TP-D	Per-operation framework ranking vs wrk2 single winner
Improved decision-making vs microbenchmarks	All TPs	Side-by-side wrk2 aggregate vs slsbench per-op comparison

=== Hypothesis Confirmation Summary ===

<pandas.io.formats.style.Styler at 0x7665e3f08a00>

Results: 6 confirmed, 0 not confirmed, 0 pending

1.8 Decision Comparison: Microbenchmark vs Scenario-Based Insights

The following contrasts illustrate how the choice of benchmarking tool affects the conclusions drawn.

1.8.1 Data Validity

- **wrk2:** 16,921 non-2xx responses out of 1,096,614.0 total (1.5% error rate). Hardcoded IDs in Lua scripts produce stale-state errors under concurrent load.

- **slsbench:** 15,320 non-2xx responses out of 1,236,586 total (1.2% error rate). Schemathesis-generated stateful chains maintain valid request payloads across all transitions.
- **Impact:** wrk2's 1.5% error rate vs slsbench's 1.2% demonstrates that stateful chain generation significantly improves data validity.

1.8.2 Framework Selection

- **wrk2 conclusion:** Reports a single aggregate latency per framework. Winner: go (lowest flow completion time).
- **slsbench conclusion:** Per-operation breakdown reveals that framework advantage depends on the operation type. Different operations may favor different frameworks, enabling workload-dependent recommendations.
- **Impact:** A team choosing a framework based solely on wrk2 aggregates could select one that is slower for their dominant workload pattern.

1.8.3 Cold-Start Analysis

- **wrk2 conclusion:** Reports one aggregate cold penalty for the entire flow.
- **slsbench conclusion:** Per-operation cold penalties vary significantly. Write operations (e.g., addOwner) suffer much higher cold penalties than read operations (e.g., listVets), guiding targeted optimization.
- **Runtime comparison:** Quarkus Native first response: 962ms vs Quarkus JVM: 12,476ms (13.0x slower startup).

1.9 6. Full Benchmark Commands

The cells above work with whatever data exists in `results/`. To populate the full evaluation dataset, execute the commands below from the `evaluation/` directory.

Prerequisites: 1. Docker images for the active runtime matrix (four benchmark applications, all sharing a pinned PostgreSQL 14 sidecar): - `aape2k/spring-petclinic-rest:v2.0.0` (Spring Boot JVM + PostgreSQL, rebuilt from v1.0.0 which used in-memory H2) - `aape2k/quarkus-petclinic:v1.0.0` (Quarkus GraalVM native) - `aape2k/quarkus-petclinic-jvm:v1.0.0` (Quarkus on the JVM) - `aape2k/go-petclinic:v1.0.0` (Go with the chi router and GORM, distroless base)

A fifth runtime, **Spring Boot (GraalVM Native)**, is scaffolded in the repository (`benchmark-app/spring-petclinic-rest/Dockerfile.native`, `evaluation/docker/spring-native-bench.yml`, runner-script branching) but deferred to a follow-up campaign: the Spring Boot 4 + GraalVM CE native-image analysis stage requires more RAM than the reference host can reliably provide. Users with a larger builder can produce `aape2k/spring-petclinic-rest-native:v1.0.0` and rerun the matrix with `--apps spring,spring-native,quarkus,quarkus-jvm,go`. 2. Tooling images: `eval-wrk2:latest`, `aape2k/slsbench:v3.0.0`, `aape2k/wrk2-flow:v3.0` 3. Probe-bodies (runtimes can share each other's probe cache where the API surface is identical; `quarkus-jvm` reuses `quarkus` flows and, when enabled, `spring-native` reuses `spring` flows): `bash ./scripts/run-probe-all.sh --apps spring,quarkus,quarkus-jvm,go --scenarios read-heavy,mixed,lifecycle`

Go full-matrix extension. At the time of this release the `go` runtime has been executed across the full matrix ($3 \text{ scenarios} \times 2 \text{ phases} \times 4 \text{ rates} \times 3 \text{ repetitions} \times 2 \text{ treatments} = 144 \text{ runs}$), while the JVM/GraalVM-native anchors still populate the earlier two-rate matrix (200 and 500 req/s). This asymmetry is why the `go` row in the auto-detected inventory reports roughly twice as many runs as the JVM/GraalVM-native rows. Cross-runtime contrasts are interpreted on the shared rate slice (200 and 500 req/s); `go`'s 50 and 1000 req/s observations are shown as part of the non-JVM-native saturation curve. To reproduce the `go` slice end-to-end:

```
./scripts/run-probe-all.sh --apps go --scenarios read-heavy,mixed,lifecycle
./scripts/run-all.sh --apps go \
  --scenarios read-heavy,mixed,lifecycle \
  --phases cold,steady \
  --reps 3 \
  --rates "50 200 500 1000"
```